Practicle 05

AIM :

Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries

- Scatter Plot
- Line Plot
- Histogram
- Box Plot
- Pair Plot

Theory :

1. Importance of Data Visualization in Data Analysis & EDA

Data visualization plays a crucial role in data analysis as it helps transform complex numerical data into visual formats that are easy to understand. It allows analysts to quickly identify patterns, trends, and relationships within the data that might not be obvious in raw form. In Exploratory Data Analysis (EDA), visualization is especially important because it helps in detecting outliers, understanding distributions, and uncovering hidden insights before applying machine learning models. Graphical representations like scatter plots, histograms, and box plots provide intuitive ways to explore data and make informed decisions.

2. Difference Between Matplotlib and Other Visualization Libraries

Matplotlib is a foundational plotting library in Python that provides extensive control over plot customization and supports a wide variety of static, animated, and interactive visualizations. Compared to other libraries like Seaborn or Plotly, Matplotlib is more low-level, meaning it requires more code but offers greater flexibility. Seaborn is built on top of Matplotlib and provides high-level functions for statistical visualization with better default styles. Plotly, on the other hand, focuses on interactive visualizations suitable for web applications. Thus, Matplotlib is best for detailed customization, while others prioritize ease of use and aesthetics.

3. Purpose of the Pyplot Module in Matplotlib

The pyplot module in Matplotlib provides a simple interface for creating and managing plots. It mimics MATLAB-style plotting and allows users to create figures, add plots, label axes, and display results using straightforward commands. Pyplot handles tasks like figure creation, plotting lines, adding titles, and showing grids,

making it easier for beginners to work with visualizations without dealing with complex object-oriented structures. Essentially, it acts as the main entry point for most plotting tasks in Matplotlib.

4. Process of Creating a Basic Line Plot

Creating a basic line plot in Matplotlib involves several steps. First, the required library (matplotlib.pyplot) is imported. Then, data for the x-axis and y-axis is defined. After that, the plot() function is used to create the line plot. Labels for the axes and a title are added to make the plot more informative. Optionally, a grid can be included for better readability. Finally, the plot is displayed using the show() function. This simple process allows users to visualize trends or changes in data over time.

5. Importance of Customization in Data Visualization

Customization is important in data visualization because it improves clarity, readability, and presentation quality. Without proper customization, plots may look confusing or fail to communicate the intended message effectively. Matplotlib provides many customization options such as changing colors, line styles, markers, labels, titles, font sizes, legends, and grid styles. Users can also adjust figure size, axis limits, and annotations to highlight important data points. These features help create visually appealing and informative graphs tailored to specific needs.

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

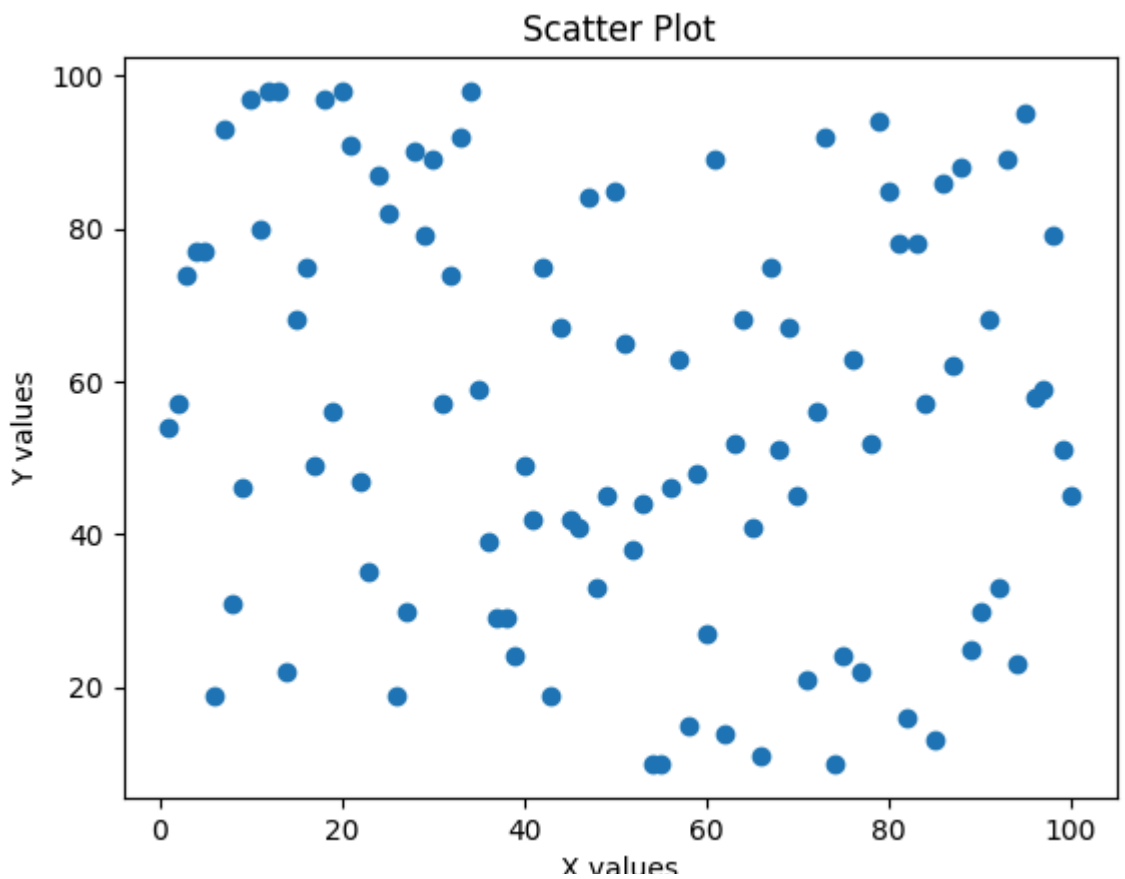
Create Sample Dataset

```
# Creating a sample dataset
np.random.seed(0)

data = pd.DataFrame({
    'X': np.arange(1, 101),
    'Y': np.random.randint(10, 100, 100),
    'Category': np.random.choice(['A', 'B', 'C'], 100)
})

data.head()
```

	X	Y	Category
0	1	54	A
1	2	57	B
2	3	74	C
3	4	77	A
4	5	77	B

Scatter Plot

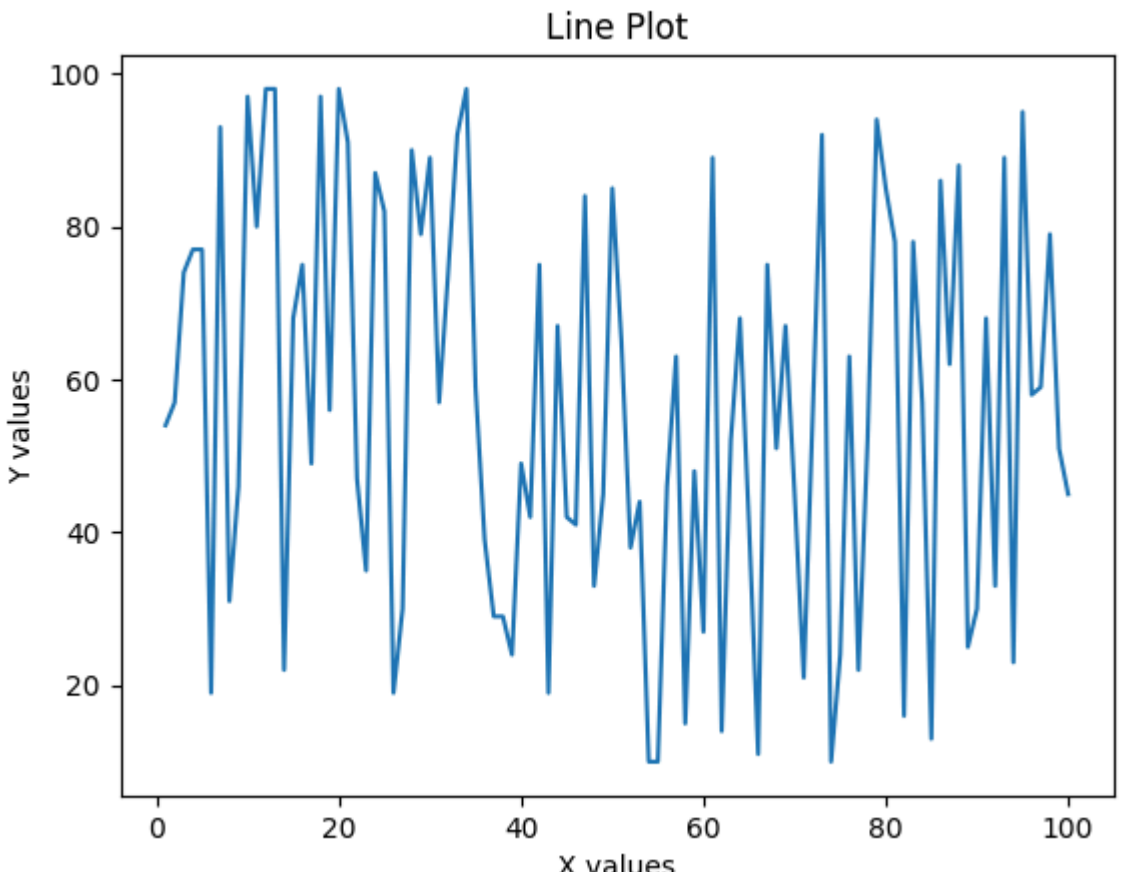
```
plt.figure()
plt.scatter(data['X'], data['Y'])
plt.title("Scatter Plot")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```



Line Plot

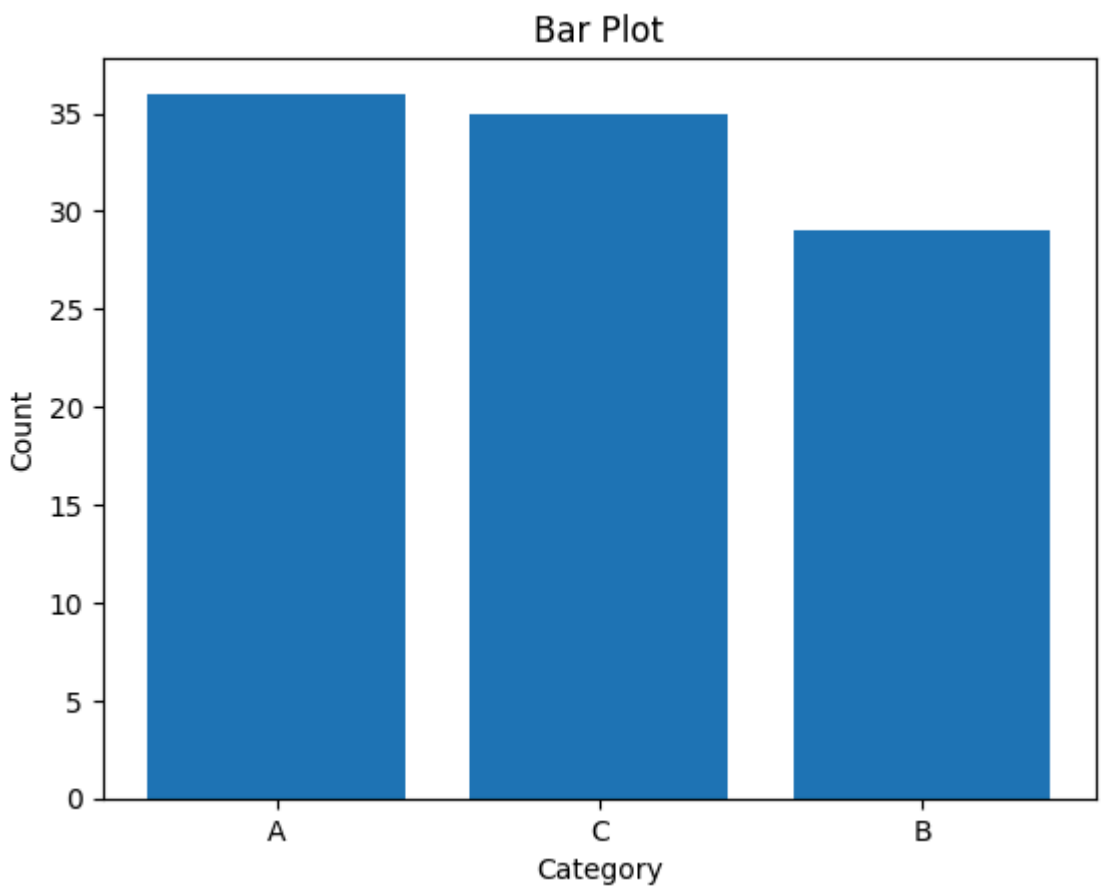
```
plt.figure()
plt.plot(data['X'], data['Y'])
plt.title("Line Plot")
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
plt.show()
```



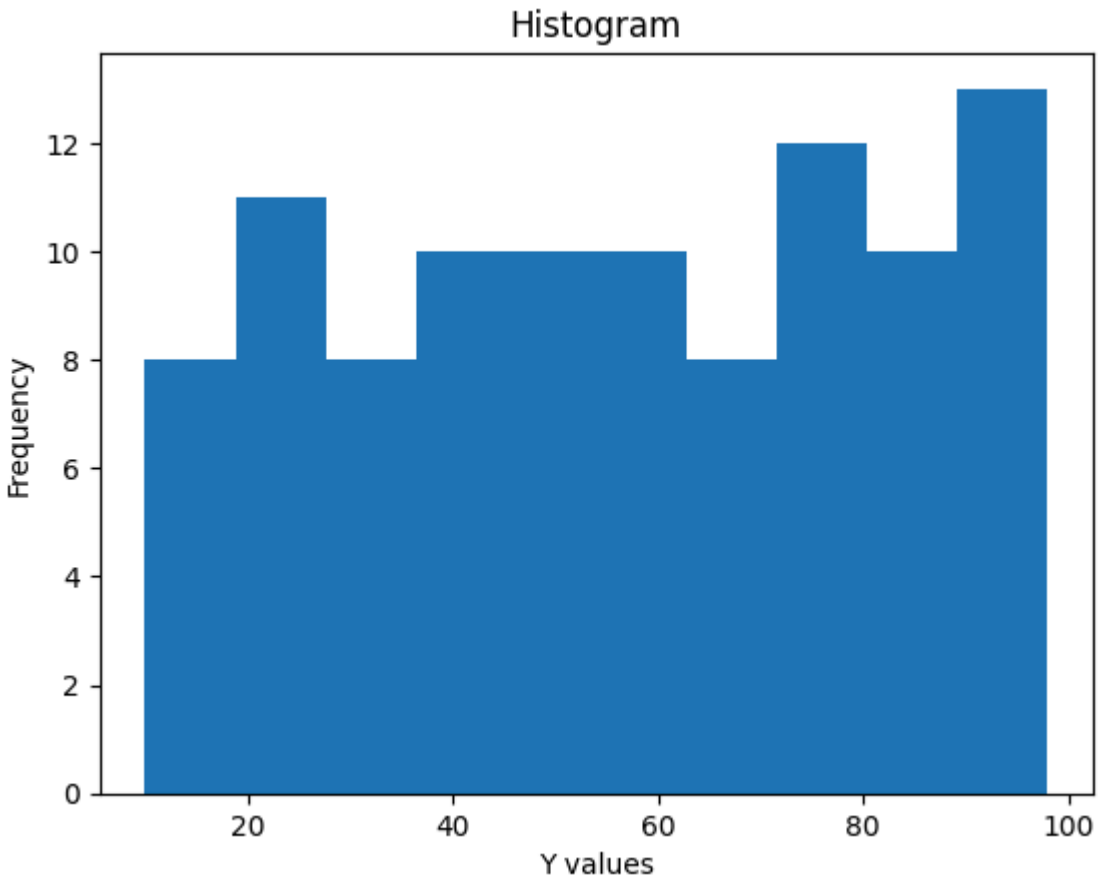
Bar Plot

```
category_counts = data['Category'].value_counts()

plt.figure()
plt.bar(category_counts.index, category_counts.values)
plt.title("Bar Plot")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```



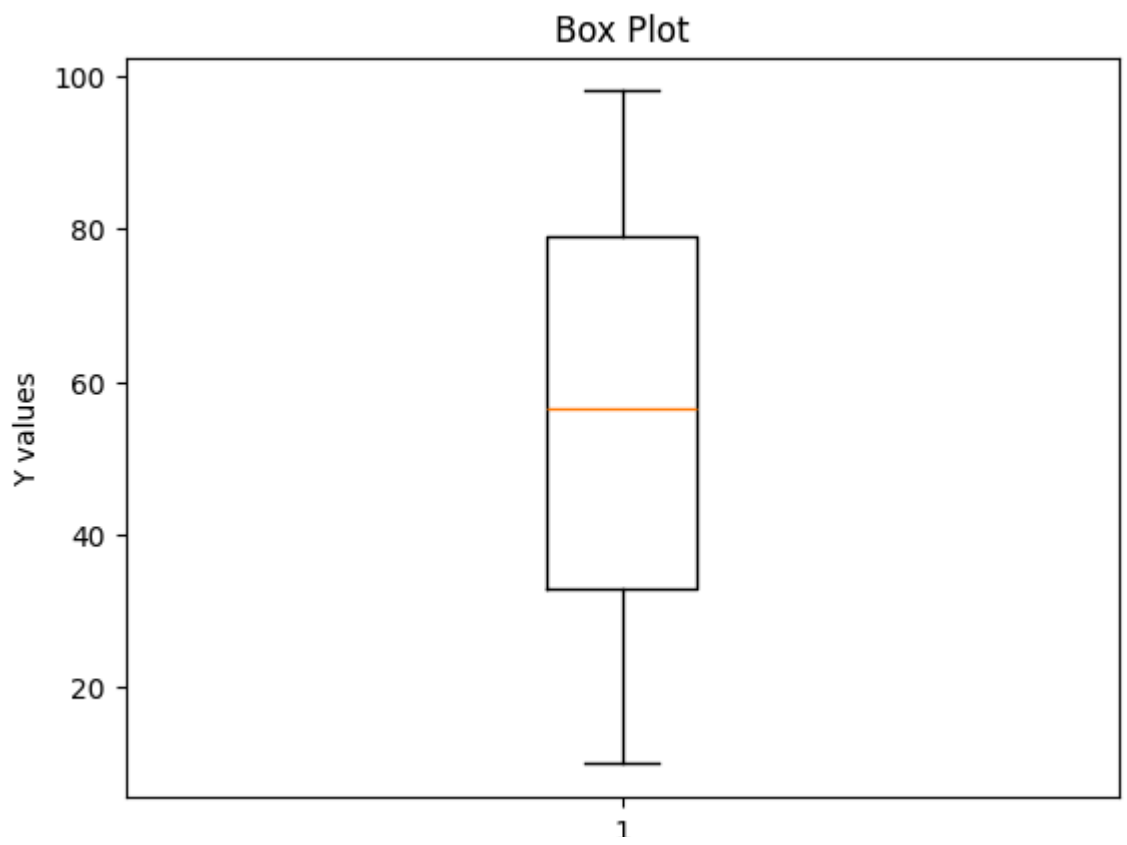
Histogram

```
plt.figure()
plt.hist(data['Y'])
plt.title("Histogram")
plt.xlabel("Y values")
plt.ylabel("Frequency")
plt.show()
```



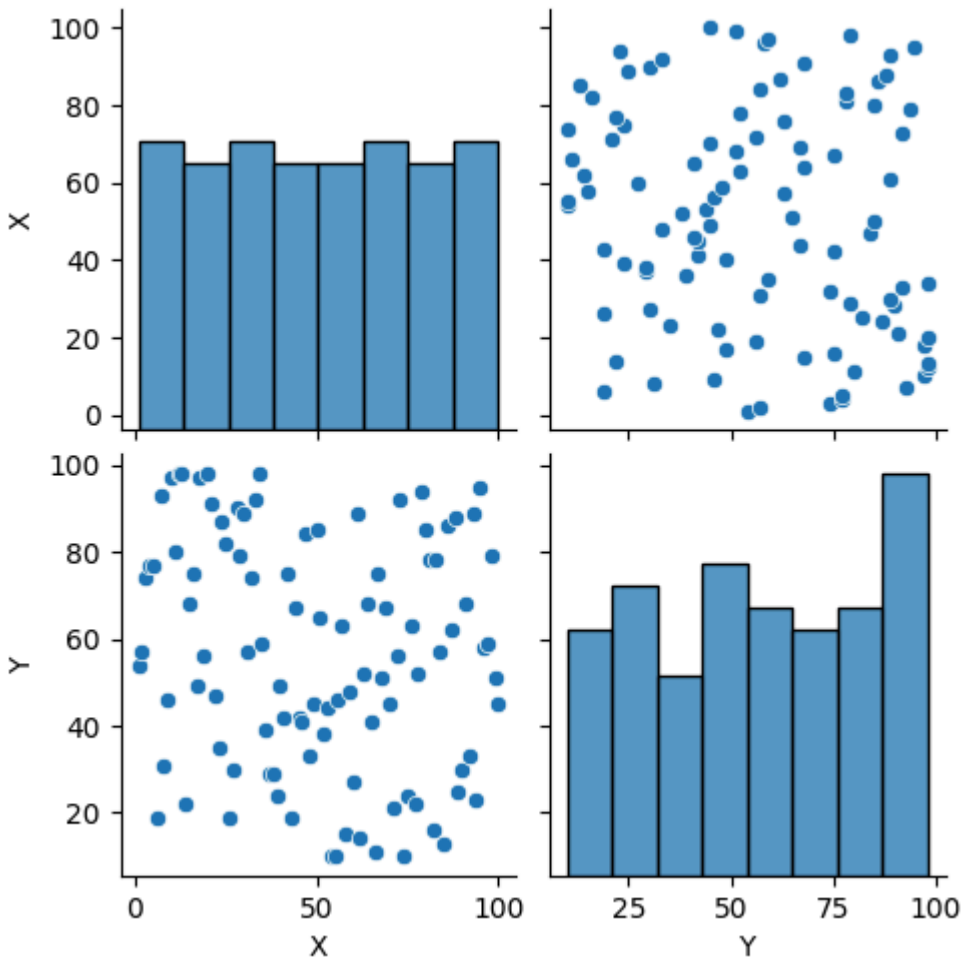
Box Plot

```
plt.figure()
plt.boxplot(data['Y'])
plt.title("Box Plot")
plt.ylabel("Y values")
plt.show()
```



Pair Plot

```
sns.pairplot(data)
plt.show()
```



COnclusion : This experiment demonstrates how data visualization techniques using Matplotlib and Seaborn help in understanding datasets effectively. Different plots such as scatter, line, bar, histogram, box plot, and pair plot provide various perspectives of the data, making it easier to analyze patterns, distribution and relationship.

IRIS DATASET

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Iris.csv to Iris.csv
```

```
data = pd.read_csv('Iris.csv')
data.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris setosa
1	2	4.9	3.0	1.4	0.2	Iris setosa
2	3	4.7	3.2	1.3	0.2	Iris setosa
3	4	4.6	3.1	1.5	0.2	Iris setosa
4	5	5.0	3.6	1.4	0.2	Iris setosa

Basic Info

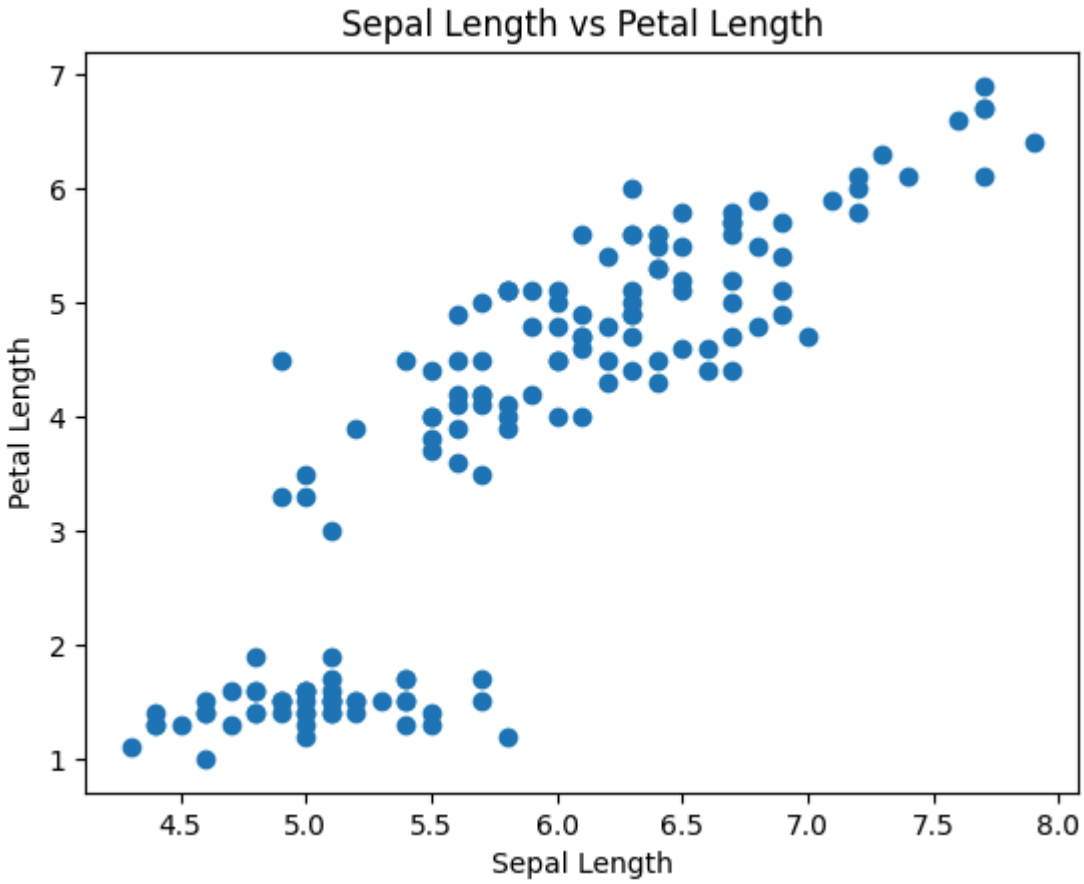
```
data.info()
data.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidth
count	150.000000	150.000000	150.000000	150.000000	150.0000
mean	75.500000	5.843333	3.054000	3.758667	1.1986
std	43.445368	0.828066	0.433594	1.764420	0.7631
min	1.000000	4.300000	2.000000	1.000000	0.1000
25%	38.250000	5.100000	2.800000	1.600000	0.3000
50%	75.500000	5.800000	3.000000	4.350000	1.3000
75%	112.750000	6.400000	3.300000	5.100000	1.8000
max	150.000000	7.900000	4.400000	6.900000	2.5000

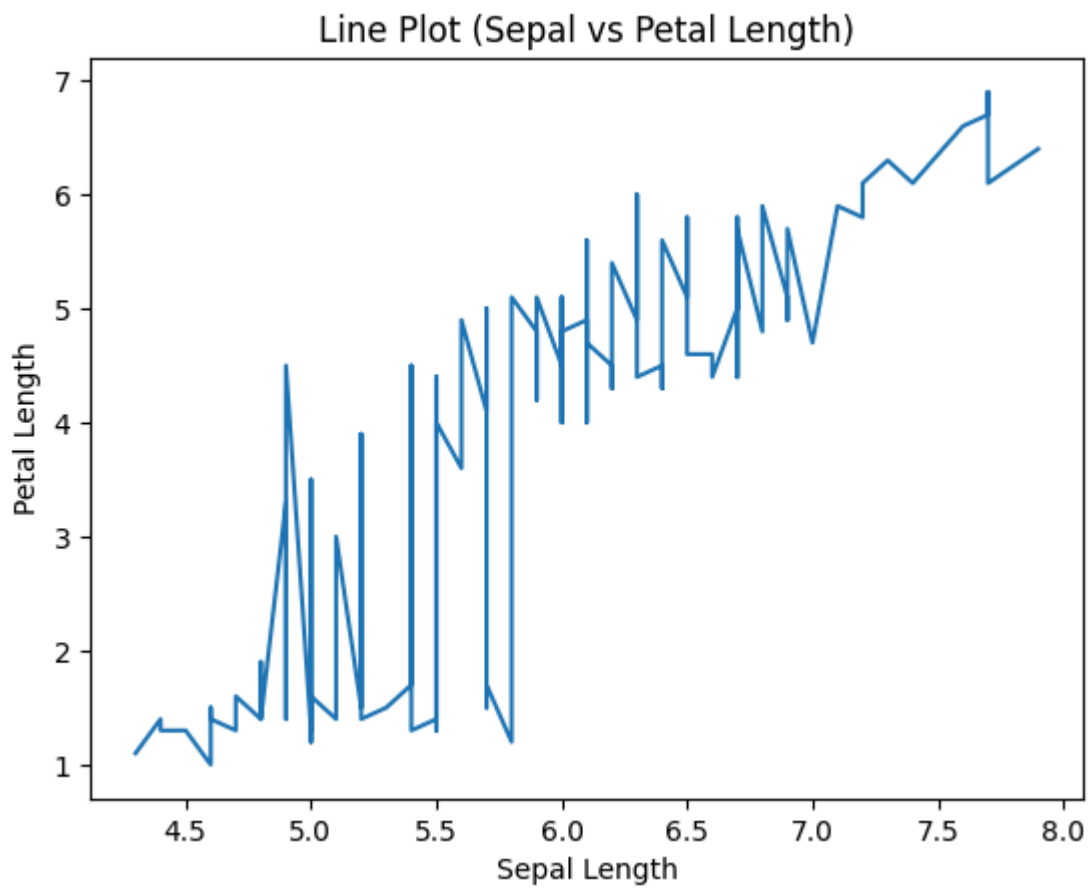
Scatter Plot

```
plt.figure()
plt.scatter(data['SepalLengthCm'], data['PetalLengthCm'])
plt.title("Sepal Length vs Petal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()
```



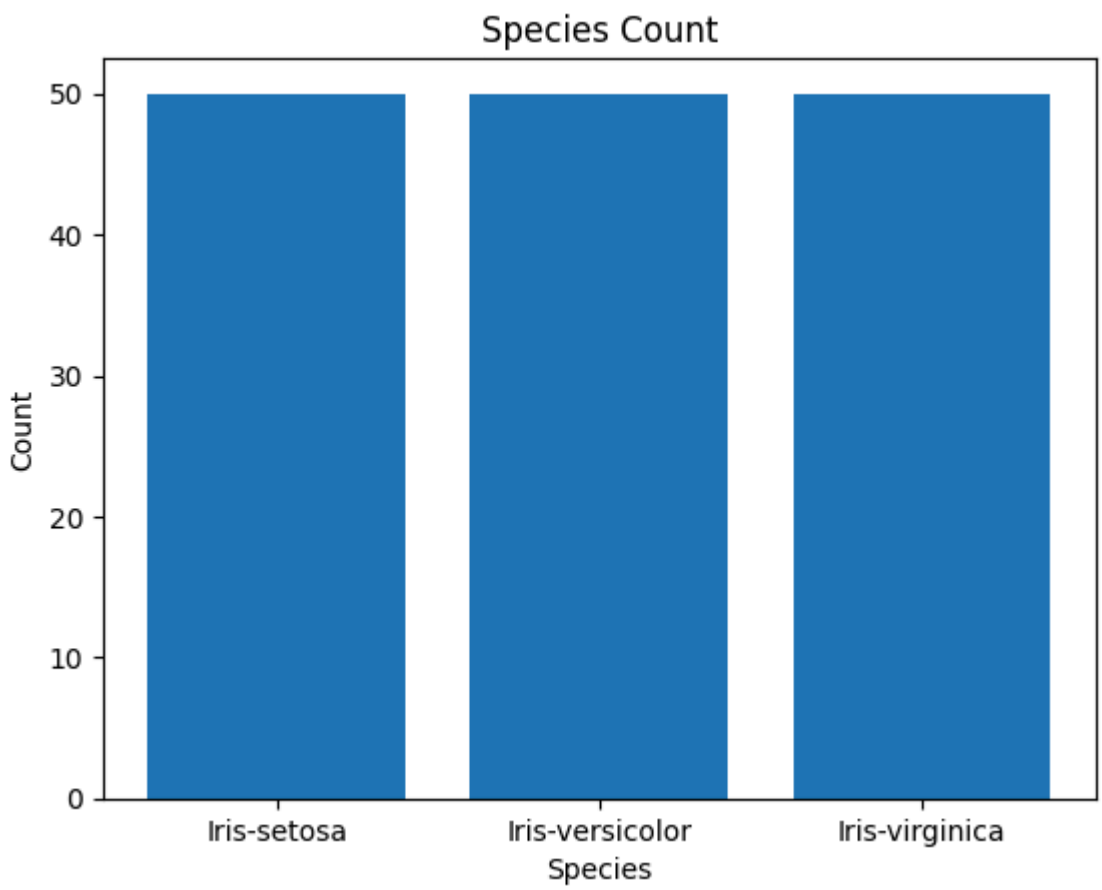
Line Plot

```
sorted_data = data.sort_values(by='SepalLengthCm')

plt.figure()
plt.plot(sorted_data['SepalLengthCm'], sorted_data['PetalLengthCm'])
plt.title("Line Plot (Sepal vs Petal Length)")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()
```



Bar Plot

```
species_count = data['Species'].value_counts()

plt.figure()
plt.bar(species_count.index, species_count.values)
plt.title("Species Count")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()
```



Histogram

```
plt.figure()
plt.hist(data['SepalLengthCm'])
plt.title("Sepal Length Distribution")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.show()
```

Sepal Length Distribution